

Django Backend Learning — Phase 3 Notes

Django REST Framework fundamentals through ModelViewSet and PUT vs PATCH

1. Why DRF?

Django REST Framework provides a structured toolkit for building APIs, including serializers, API views, generic views, viewsets and routers.

2. Serializers

A serializer converts Django/Python data into API-friendly data and validates incoming data before saving it.

```
Model object ↔ Serializer ↔ JSON/API data
```

ModelSerializer can derive many fields from a Django model.

3. Validation

We created field-level validation such as `validate_price()` to reject invalid data before saving.

```
def validate_price(self, value):
    if value <= 0:
        raise serializers.ValidationError('Price must be greater than 0.')
    return value
```

4. APIView

APIView lets you explicitly define behavior for HTTP methods such as `get()`, `post()`, `patch()` and `delete()`.

```
class ProductView(APIView):
    def get(self, request):
        ...

    def post(self, request):
        ...
```

5. Generic Views

Generic views provide common API behavior without requiring you to write all CRUD logic manually.

```
class ProductListView(ListCreateAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

ListCreateAPIView handles list/create behavior. RetrieveUpdateDestroyAPIView handles retrieve/update/destroy behavior.

6. ViewSets

A ViewSet groups operations for one resource. Common actions include `list()`, `create()`, `retrieve()`, `partial_update()` and `destroy()`.

```
GET /products/ → list()
POST /products/ → create()
GET /products/1/ → retrieve()
PATCH /products/1/ → partial_update()
```

```
DELETE /products/1/ → destroy()
```

7. Routers

A Router generates URL patterns for a ViewSet and reduces URL configuration boilerplate.

```
router = DefaultRouter()
router.register('products', ProductViewSet, basename='product')
urlpatterns = router.urls
```

Mental model: ViewSet defines actions; Router connects URLs to those actions.

8. ModelViewSet

ModelViewSet provides standard CRUD actions for a model, which is why the Product API became very small.

```
class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

It provides list(), create(), retrieve(), update(), partial_update(), and destroy().

9. PUT vs PATCH

PUT is conceptually a full update/replacement. Required fields generally need to be supplied.

PATCH is a partial update. Only fields being changed need to be supplied.

```
PATCH /products/1/
{
  "price": 1000
}
```

In DRF, PATCH is handled as a partial update, conceptually using partial=True with the serializer.

10. DRF Request Flow

A useful mental model is:

```
Client → URL/Router → View/ViewSet → Serializer → Validation → ORM/Model → Database → Response
```

Each layer has a job. This separation is one of the most important backend concepts from this phase.

11. Revision Checklist

- I understand the purpose of DRF.
- I understand serializers and validation.
- I understand APIView.
- I understand Generic Views.
- I understand ViewSets and Routers.
- I understand ModelViewSet.
- I understand PUT vs PATCH.
- I can trace a basic DRF request from URL to database and back.